

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	K.BHAVANA	Reg. No.:	192524178
Date:	21-08-2026	Duration:	60 minutes

Code of Conduct

I K.BHAVANA (192524178) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

K. Bhavana

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Case Study 33 – SDG 5:

Women Entrepreneurship Support Portal

Many women-owned small businesses struggle to access government schemes, financial assistance, and business mentoring. Existing information is fragmented across multiple websites. A government agency plans to create an integrated digital platform supporting registration, funding opportunities, training programs, and networking. As a software engineer, analyze the above problem and propose a suitable software engineering solution.

1. Introduction

This document is the CO4 Assessment Tool 2 deliverable for Case Study 33 – SDG 5: Women Entrepreneurship Support Portal, and follows on directly from the CO4 Assessment Tool 1 Test Case Design report submitted previously for the same system. Where Assessment Tool 1 focused on verifying the correctness, functionality, reliability and usability of the portal through manually designed test cases, this Assessment Tool 2 report focuses on automating the software delivery lifecycle itself — that is, on designing a Continuous Integration and Continuous Deployment (CI/CD) pipeline that builds, tests, quality-checks, packages and deploys the same Women Entrepreneurship Support Portal in a repeatable, auditable and low-risk manner.

A CI/CD pipeline is the backbone of modern DevOps practice: it takes the manual, error-prone steps of compiling code, running tests, checking code quality and pushing a build to a server, and turns them into a single, automated, version-controlled workflow that runs identically every time a developer pushes a change. For a government-facing platform such as the Women Entrepreneurship Support Portal — which handles sensitive Udyam registration numbers, turnover figures, funding applications and personally identifiable information for a vulnerable user segment — a disciplined CI/CD pipeline is not a convenience but a necessity: it is the mechanism through which the 53 test cases, security controls and non-functional requirements identified in the previous technical report are continuously and automatically re-verified on every single code change, rather than relying on ad-hoc manual testing before each release.

This report directly addresses CO4: “Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics,” by translating the test case design work of Assessment Tool 1 into an executable DevOps pipeline that runs those checks automatically, gates every deployment behind passing quality and security checks, and provides a safe, reversible path from a developer's local commit to a live production environment serving women entrepreneurs, certified mentors and government officers.

1.1 Purpose and Scope of this Document

The purpose of this document is to design — not to execute — a complete CI/CD pipeline for the Women Entrepreneurship Support Portal. It covers:

- Analysis of the CI/CD needs arising from the system's architecture, requirements and prior test case design.
- A staged pipeline workflow from source code management through to production deployment.
- A justified toolchain selection for every stage of the pipeline.
- An automated testing strategy that operationalizes the 53 test cases and requirement traceability matrix from Assessment Tool 1.
- A three-environment deployment strategy (Development, Staging, Production) with a blue-green production release model.

- Security scanning, code-quality gates, notification mechanisms and a rollback strategy.
- A complete pipeline diagram and a step-by-step narrative from code commit to live deployment.

As with the previous technical report, this design is grounded in the architecture and issues already documented for the portal (React 18/Vite frontend, Node.js/Express REST API, SQLite via sql.js data layer, Docker-based packaging) rather than in a generic, system-agnostic CI/CD template.

2. Problem Statement Recap

Case Study 33 – SDG 5: Women Entrepreneurship Support Portal states that many women-owned small businesses struggle to access government schemes, financial assistance and business mentoring because existing information is fragmented across multiple websites. A government agency therefore plans to create an integrated digital platform supporting registration, funding opportunities, training programmes and networking. The Women Entrepreneurship Support Portal, as documented in the previous technical report and re-verified through the Assessment Tool 1 test case design, is the realised solution to this problem: a role-based, full-stack web application serving three actors — the Women Entrepreneur, the Certified Mentor and the Government Officer.

For this Assessment Tool 2 exercise, the objective shifts from verifying the correctness of that system to designing how the system's source code, tests and container images move automatically and safely from a developer's machine into a running production environment. The underlying technical facts about the system — its three-tier architecture, its 22 functional requirements, its 10 non-functional requirements and its existing Docker/Docker Compose based packaging — are carried forward unchanged from the previous technical report and from the Assessment Tool 1 report, and are used here as the concrete basis for every pipeline design decision.

2.1 System Overview (carried forward from the previous technical report)

The Women Entrepreneurship Support Portal is a centralized, role-based platform with three actors, each provided a dedicated dashboard:

- Women Entrepreneur: registers a business profile (Udyam number, sector, turnover, employee count), browses and searches government schemes and funding opportunities, applies for funding through a multi-step form, tracks application status, enrolls in training workshops, and requests mentorship.
- Certified Mentor: manages profile and availability, reviews and accepts/rejects mentorship requests, and conducts advisory sessions with meeting links and feedback capture.
- Government Officer: audits and verifies registered businesses, manages scheme/funding/training/networking catalogues, reviews funding applications, manages announcements and user accounts, and generates filterable CSV reports.

2.1.1 Technology Architecture

The system follows a three-layer architecture, which is the direct input to every CI/CD tooling decision made in Section 6 of this report:

- Presentation Layer: React 18 single-page application built with Vite, using React Router and Recharts.
- Application & API Layer: Node.js and Express.js REST API, with JWT-based authorization middleware restricting access by role.
- Data Access Layer: a WebAssembly-compiled SQLite engine (sql.js) with disk-synchronized persistence, comprising sixteen normalized relational tables.

The request flow is: React/Vite frontend → Node.js/Express REST API → SQLite (sql.js) data layer, deployed via Docker. This monolithic-container, single-repository structure (confirmed by the git status output reproduced later in this report as Fig. 4, which shows client/, server/ and Docker artifacts committed together) directly shapes the pipeline design: a single build job can compile both the frontend and backend, and a single Docker image can be produced, versioned and promoted across environments.

3. CI/CD Needs Analysis

Before designing the pipeline itself, the specific pressures that make CI/CD necessary for this particular system were analysed against the requirements and issues already documented in the previous technical report and the Assessment Tool 1 test case design. Five needs stood out.

3.1 Need for Continuous, Automated Regression of a Large Test Suite

Assessment Tool 1 designed 53 detailed test cases (TC-01 to TC-53) across registration, login, business profile, verification, schemes, funding, training, networking, mentorship, dashboards, security, exceptional handling, usability, announcements, notifications, reports and user management, all of which were left in an explicit “Not Executed” state pending formal execution. Running 53 test cases manually before every release is slow, inconsistent between testers, and does not scale as the codebase grows. This is the single strongest driver for CI: a pipeline is needed that automatically executes the equivalent of this suite — as unit, integration and end-to-end tests — on every commit, so that regressions are caught within minutes rather than discovered after deployment.

3.2 Need for Consistent, Reproducible Builds Across Three Roles and Three Layers

The portal's three-tier architecture (React/Vite frontend, Node/Express API, sql.js data layer) and three distinct role-based front-ends (Entrepreneur, Mentor, Officer) mean a build is only valid if all three roles' UI bundles compile correctly against a backend that itself starts cleanly against a fresh, migrated database. Manually verifying this on a developer's laptop does not guarantee the same result on a teammate's machine or on the production server. A CI pipeline that builds inside a clean, disposable container on every commit removes “works on my machine” risk.

3.3 Need for Enforced Security and Access-Control Verification

Section 6 of the previous test case report defines strict role boundaries (an Entrepreneur must never reach /officer/* or /mentor/* routes; a Mentor must never reach /officer/* routes), and Assessment Tool 1 dedicates five test cases (TC-08, TC-37, TC-41, TC-42, TC-43) purely to authentication and authorization security. Because the portal stores Udyam registration numbers, turnover figures and funding application data for a vulnerable user population, a regression in JWT validation or role-based middleware is a security incident, not just a bug. CI/CD is needed to run these authorization checks, dependency vulnerability scans and secret-leak scans automatically and to block any merge or deployment that fails them.

3.4 Need for Safe, Reversible Production Releases

The state-transition analysis in the previous report (Section 8.4) shows that Business Profile, Funding Application and Mentorship Request are all stateful entities whose invalid transitions must be guarded against (e.g., TC-17: re-approving an already-verified business must not corrupt state). A bad deployment that reaches production and is only then discovered to break one of these state machines could leave real business or funding records in an inconsistent state. CI/CD is needed to provide a deployment strategy — covered in Section 8 of this report — that can detect a bad release quickly via automated smoke tests and roll it back within minutes without manual, error-prone intervention.

3.5 Need to Close the Non-Functional Requirement Gaps Identified in Assessment Tool 1

The Requirements Traceability Matrix in Assessment Tool 1 explicitly flagged NFR10 (Responsiveness) as Not Covered by any test case, and NFR2 (Performance), NFR5 (Maintainability) and NFR6 (Scalability) as only Partially covered through design-level review rather than executed tests. A well-designed CI/CD pipeline is the natural place to close these gaps: automated responsive-viewport testing, bundle-size/performance budgets, and static architecture/dependency checks can all be added as pipeline stages without requiring additional manual QA effort. This linkage is made explicit in the Traceability Matrix in Section 13 of this report.

3.6 Summary of CI/CD Requirements

Requirement Area	Driving Evidence (from AT1 / Technical Report)	CI/CD Capability Needed
Regression safety	53 test cases (TC-01–TC-53), all “Not Executed”	Automated unit/integration/E2E test execution on every commit
Build consistency	3-tier architecture, 3 role-based UIs	Clean, containerized, reproducible

Requirement Area	Driving Evidence (from AT1 / Technical Report)	CI/CD Capability Needed
		build stage
Security enforcement	TC-08, TC-37, TC-41–TC-43; NFR1	Automated RBAC tests, SAST, dependency & secret scanning
Safe releases	State-transition rules (Business Profile, Funding, Mentorship)	Staged deployment, smoke tests, blue-green cutover, rollback
NFR coverage gaps	NFR10 not covered; NFR2/5/6 partial (AT1 RTM)	Responsive/performance/architecture checks as pipeline gates
Auditability	Code of Conduct / academic integrity, government platform	Pipeline run history, approver identity, audit logging

Table 1: Summary of CI/CD requirements derived from prior analysis of the Women Entrepreneurship Support Portal.

4. CI/CD Pipeline Architecture and Workflow

The pipeline is organized as eight sequential stages, plus five cross-cutting concerns that apply across every stage rather than at a single point. Figure 1 shows the complete architecture; each stage is then described in detail.

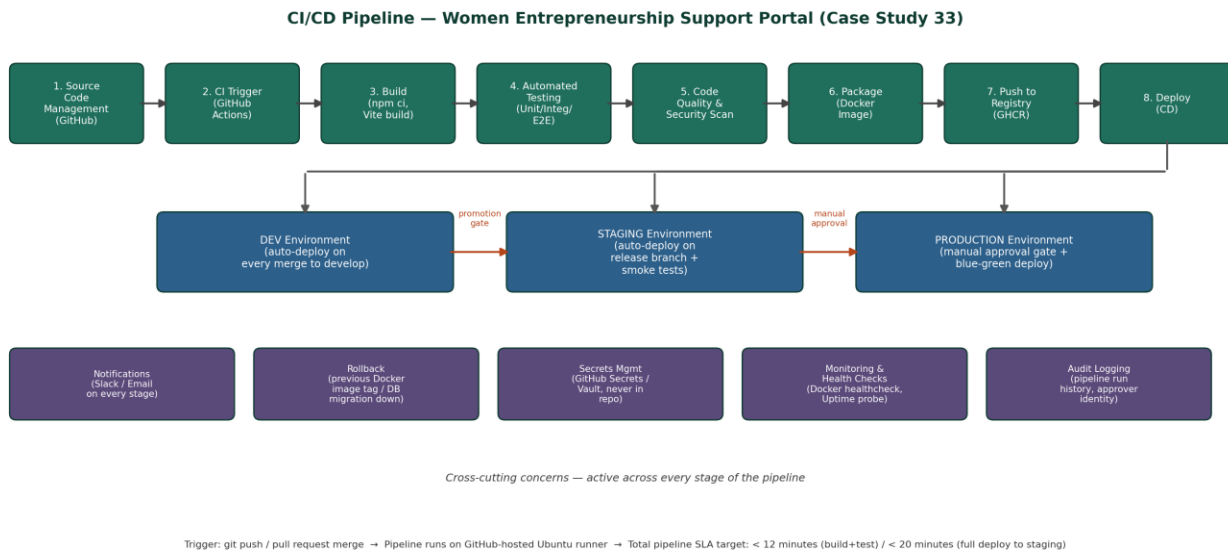


Fig. 1: End-to-end CI/CD pipeline architecture for the Women Entrepreneurship Support Portal, showing the eight sequential stages, the three deployment environments, and the five cross-cutting concerns.

4.1 Stage-by-Stage Description

Stage 1 – Source Code Management

All source code (client/, server/, Dockerfile, docker-compose.yml) resides in a single Git repository hosted on GitHub, matching the actual repository structure evidenced by the git status output in Fig. 4. Three long-lived/short-lived branch types are used: develop (integration branch), release/* (stabilization branch per release), and feature/* / hotfix/* (working branches). Branch protection rules require at least one peer review and a passing CI status check before any merge into develop, release/* or main.

Stage 2 – CI Trigger

A GitHub Actions workflow is triggered automatically on: (a) every push to a feature/* or hotfix/* branch, (b) every pull request opened against develop, release/* or main, and (c) every merge/push directly to develop, release/* or main. This ensures that code is validated both before and after it is integrated, and that no commit reaches a shared branch without having passed the pipeline at least once.

Stage 3 – Build

The build stage installs dependencies with npm ci (a clean, lockfile-exact install, chosen over npm install for build reproducibility), then runs two parallel build steps: a Vite production build of the client/ React application, and a syntax/type check of the server/ Express application. Build artifacts (the client dist/ bundle) are cached as GitHub Actions artifacts so that later stages do not need to rebuild them.

Stage 4 – Automated Testing

Three layers of automated tests run in this stage: unit tests (Jest) for isolated business logic such as Udyam duplicate detection and equivalence-partition validation rules; integration tests (Jest + Supertest) that exercise real Express routes against a disposable, freshly seeded sql.js test database; and end-to-end tests (Playwright) that drive the built React application through full user journeys for all three roles. The complete strategy, including explicit mapping back to the 53 Assessment Tool 1 test

cases, is given in Section 6.

Stage 5 – Code Quality and Security Analysis

Static analysis runs ESLint (JavaScript/JSX style and correctness rules) and a code-coverage check (minimum 70% line coverage, enforced as a merge gate). Security analysis runs GitHub CodeQL (SAST) for injection and authentication-bypass patterns, npm audit / Dependabot for known-vulnerable dependencies, and Gitleaks to catch any accidentally committed secret (JWT signing key, database credentials) before it reaches a shared branch.

Stage 6 – Packaging

A multi-stage Dockerfile builds a single production image containing the compiled React bundle served by the Express application (matching the existing Dockerfile/docker-compose.yml already present in the repository, as seen in Fig. 5). The image is tagged twice: once with the short Git commit SHA (immutable, for traceability) and once with the semantic version derived from the release branch (e.g., v1.2.0), so that any historical build can be re-deployed exactly.

Stage 7 – Push to Registry

The tagged image is pushed to the GitHub Container Registry (GHCR), which is used because it shares authentication directly with the GitHub Actions workflow (no separate registry credentials to manage) and integrates image vulnerability scanning automatically.

Stage 8 – Deploy (Continuous Deployment)

The same, already-tested and already-scanned image is progressively promoted through three environments — Development, Staging and Production — without ever being rebuilt (the “build once, deploy many times” principle). This is described fully in Section 7.

5. Tools and Technologies per Stage

Every tool selected below was chosen to match the portal's existing, already-implemented technology stack (React/Vite, Node.js/Express, sql.js, Docker) rather than introducing an unrelated toolchain, so that the CI/CD pipeline can be adopted with minimal migration effort.

Stage	Tool(s) Selected	Justification
Source Code Management	Git + GitHub	Already in use for the project (confirmed by the git status screenshot, Fig. 4); provides branch protection, pull-request review and full commit history.
CI Orchestration	GitHub Actions	Native integration with the GitHub repository — no separate CI server to host or secure; free minutes for small teams; YAML workflow files are version-controlled alongside the code they build.
Dependency Install / Build	npm ci, Vite build	npm ci performs a clean, lockfile-exact install for reproducible builds; Vite is the build tool already used by the React 18 client.
Backend Runtime	Node.js 18 LTS	Matches the Express.js REST API already implemented; LTS release chosen for long-term security patch support.
Unit Testing	Jest	Industry-standard test runner for both React components (with React Testing Library) and Node/Express service functions; fast, parallelized, well documented.
Integration / API Testing	Jest + Supertest	Supertest allows HTTP-level assertions against the Express app in-process, without needing a deployed server, exercising real route handlers and JWT middleware.
End-to-End Testing	Playwright	Can drive a real browser through full multi-step user journeys (registration, funding application, mentorship) across all three role-based UIs; supports parallel, cross-browser execution.
Static Analysis / Linting	ESLint + Prettier	JavaScript/JSX-native tooling; enforces consistent style and catches common bugs (unused variables, unreachable code) before they reach code review.
SAST (Security Static Analysis)	GitHub CodeQL	Free for GitHub-hosted repositories; detects injection, authentication-bypass and unsafe-deserialization patterns directly in pull requests.
Dependency Vulnerability Scanning	npm audit + Dependabot	Flags known-CVE packages in package-lock.json and automatically opens update pull requests, closing the loop rather than only alerting.
Secret Scanning	Gitleaks	Scans every commit for accidentally hardcoded JWT secret keys, DB credentials or API tokens before they can be pushed to a shared branch.
Containerization	Docker (multi-stage Dockerfile)	The project already ships a Dockerfile and docker-compose.yml (Fig. 5); a multi-stage build keeps the final production image small by discarding build-time dependencies.
Container Registry	GitHub Container Registry (GHCR)	Shares authentication with GitHub Actions (no extra credentials to rotate); integrates image vulnerability scanning; free for this project's scale.
Container Image Scanning	Trivy	Open-source scanner that checks the built image's OS packages and application dependencies for known

Stage	Tool(s) Selected	Justification
		vulnerabilities before the image is promoted.
Deployment Orchestration	Docker (Dev/Staging), Compose with a documented Kubernetes migration path for future scale	Matches the existing docker-compose.yml; Compose is sufficient for the portal's current single-container-per-service footprint (Fig. 6 shows a healthy Compose-managed container).
Reverse Proxy / Load Balancer	Nginx	Terminates TLS, enables the blue-green traffic-cutover mechanism described in Section 7.3, and can rate-limit public endpoints such as /api/schemes.
Monitoring / Health Checks	Docker HEALTHCHECK against /api/schemes, Uptime Kuma	Directly operationalizes NFR7 (Availability), which already names /api/schemes as the health-check endpoint in the previous technical report.
Notifications	Slack Incoming Webhook (primary) + Email (fallback)	Immediate, low-friction visibility of pipeline status for the whole team; email fallback ensures the on-call approver is reachable outside Slack.
Secrets Management	GitHub Encrypted Secrets	JWT signing key, DB credentials and registry tokens are injected as environment variables at run time and are never committed to the repository.

Table 2: Tools and technologies selected for each CI/CD pipeline stage, with justification tied to the portal's existing architecture.

5.1 Why Not a Heavier Toolchain?

Tools such as Jenkins, a self-managed Kubernetes cluster, or a commercial APM suite were deliberately not selected at this stage. The portal is a single-container, three-tier monolith serving a bounded, government-scale user base; introducing Kubernetes now would add operational overhead (cluster management, networking, secrets operators) without a corresponding benefit, since Docker Compose already meets the current availability and scalability requirements (NFR6, NFR7) as documented in Assessment Tool 1. The pipeline design below is deliberately left extensible — the same Docker image that Compose deploys today could be deployed to Kubernetes tomorrow with no change to Stages 1–7.

6. Automated Testing Strategy

The testing strategy converts the 53 manually designed test cases from Assessment Tool 1 into an automated, four-layer regression suite that runs inside the CI pipeline on every commit. No manual test execution is required for regression coverage once this suite is in place; manual/exploratory testing is reserved for new features not yet covered by an automated test.

6.1 Layer 1 — Unit Tests

Unit tests validate isolated business logic with no network or database dependency, directly automating the Equivalence Partitioning and Boundary Value Analysis techniques from Assessment Tool 1 Section 8.

Unit Under Test	AT1 Technique Reused	Representative Automated Assertions
Email format & uniqueness validator	Equivalence Partitioning (Section 8.1)	Rejects malformed address; rejects duplicate; accepts well-formed unique address.
Employee count validator	Boundary Value Analysis (Section 8.2)	Accepts 1 and 500; rejects 0, -1 and 501 (boundary cases from TC-11, TC-12).
Turnover validator	Boundary Value Analysis (Section 8.2)	Rejects non-numeric, negative and empty values (TC-13).
Funding-eligibility decision function	Decision Table (Section 8.3)	Reproduces rules R1–R5 exactly: verified+eligible+valid → accepted; any false input → blocked with correct error.
Business Profile state-transition guard	State Transition Testing (Section 8.4)	Re-approving an already-Verified profile is rejected/ignored without corrupting state (TC-17).
Password hashing utility	NFR1 Security	bcrypt hash never equals plaintext; verified hash round-trips correctly.

Table 3: Unit test layer, showing direct reuse of Assessment Tool 1 test design techniques.

6.2 Layer 2 — Integration Tests

Integration tests use Supertest to send real HTTP requests to the Express application, running against a freshly seeded, disposable sql.js test database that is reset before every test file to keep results deterministic. These tests automate the API-level scenarios from Assessment Tool 1, including TC-05 to TC-08 (login), TC-15 to TC-17 (officer verification), TC-23 to TC-26 (funding applications), TC-29/TC-30 (training duplicate/capacity guards), TC-32/TC-33 (networking duplicate/capacity guards) and TC-37/TC-41 to TC-43 (unauthorized access, expired tokens, missing tokens).

6.3 Layer 3 — End-to-End (E2E) Tests

Playwright drives a headless Chromium browser against the fully built application (client + server running together, exactly as it will run in Staging) to exercise complete, multi-step user journeys that a unit or integration test cannot cover on its own:

- Entrepreneur journey: register → login → complete business profile → view dashboard recommendations (TS-01, TS-03, TS-11).
- Funding journey: verified entrepreneur → multi-step funding application → officer reviews and approves → status timeline updates (TS-07, decision table rules R1–R4).
- Mentorship journey: entrepreneur sends request → mentor accepts → session with meeting link is created (TS-10).
- Cross-role security journey: an authenticated Entrepreneur token attempts to call a Mentor-only or Officer-only endpoint directly and receives HTTP 403 (TS-13, TS-15, TC-37, TC-41).

6.4 Layer 4 — Non-Functional and Regression Gates

Beyond functional correctness, the pipeline runs four additional automated checks that directly close the coverage gaps

flagged in the Assessment Tool 1 Requirements Traceability Matrix:

- Responsive-viewport tests (Playwright, multiple viewport sizes) — closes the NFR10 “Not Covered” gap.
- Bundle-size and Lighthouse performance budget checks on the built client — strengthens NFR2 coverage beyond the “observed indirectly” status in AT1.
- Dependency-graph / layering lint rule that fails the build if the presentation layer imports directly from the data-access layer — operationalizes NFR5 (Maintainability) as an automated check rather than a manual code review.
- Accessibility audit (axe-core) against key pages — automates NFR8, previously verified only by manual review.

6.5 Test Data Management

Every CI run provisions a fresh sql.js database file seeded with a small, fixed dataset (a handful of entrepreneurs, mentors, officers, schemes and funding opportunities) so that test outcomes never depend on data left over from a previous run. This mirrors the disposable, per-run philosophy already implied by the sql.js in-memory engine documented in the previous technical report, and avoids the shared-test-database flakiness that would otherwise make TC-03 (duplicate email), TC-29 (duplicate enrolment) and TC-32 (duplicate registration) unreliable to automate.

6.6 Quality Gate Policy

A pull request cannot be merged into develop, release/* or main unless all of the following are true: all unit, integration and E2E tests pass; line coverage is at least 70% (measured by Jest's coverage reporter); ESLint reports zero errors; CodeQL and Trivy report no critical or high-severity findings; and Gitleaks reports no detected secrets. This policy converts the previous report's manual “Actual Result / Pass-Fail” columns into an automatic, enforced merge gate.

7. Deployment Strategy

The deployment strategy promotes a single, immutable Docker image — built and tested exactly once in Stage 3–6 — through three progressively stricter environments. No environment ever rebuilds the application from source; each environment only ever runs the already-verified image, which guarantees that what is tested in Staging is byte-for-byte identical to what runs in Production.

7.1 Git Branching Strategy and Environment Mapping

The branching model follows a simplified GitFlow adapted to the project's scale, mapping cleanly onto the three deployment environments:

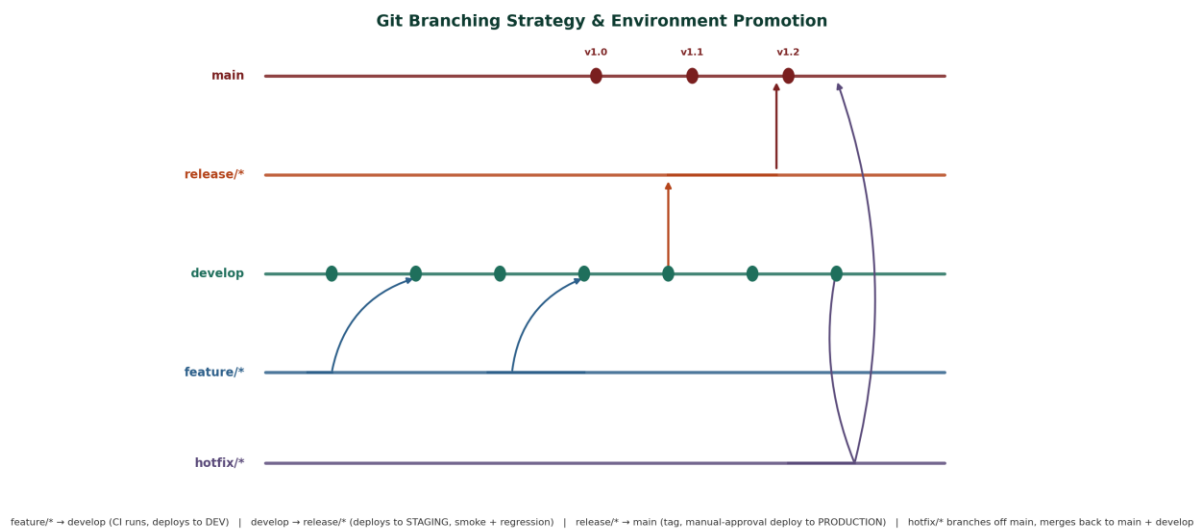


Fig. 2: Git branching strategy showing how feature, release and hotfix branches map onto the Development, Staging and Production environments.

Branch	Purpose	Auto-Deploys To	Gate Before Merge
feature/*	New functionality (e.g., feature/mentor-chat)	Not deployed — CI runs build/test/lint only	Passing CI + 1 peer review
develop	Integration branch for all completed features	Development environment	Passing CI on the feature branch's PR
release/*	Stabilization branch for an upcoming release	Staging environment	Full regression suite + smoke tests pass on Development
main	Production-ready, tagged releases only	Production environment	Full regression + smoke tests pass on Staging + manual approval
hotfix/*	Emergency fix branched directly from main	Staging, then Production (expedited)	Reduced but mandatory security + regression subset + manual approval

Table 4: Branch-to-environment mapping and merge gates.

7.2 Environment Definitions

7.2.1 Development Environment

Purpose: fast feedback for the team. Every merge to develop automatically redeploys the Development environment within minutes, using a lightweight single-container Docker Compose stack. Data is non-sensitive seed/test data only. No manual approval is required, and the environment is expected to be occasionally unstable as work in progress lands — it is explicitly

not used for demonstrations to government stakeholders.

7.2.2 Staging Environment

Purpose: a production-like rehearsal environment. Every merge to a release/* branch triggers a deployment to Staging using the same Docker image, same Docker Compose topology (backend + database + reverse proxy) and same environment-variable structure as Production, differing only in the secrets and domain name used. Immediately after deployment, an automated smoke-test job re-runs a focused subset of the E2E suite (login, one funding application, one mentorship request, the RBAC negative tests) against the live Staging URL. Only if these smoke tests pass is the release branch eligible for promotion to Production. This environment is also where the government client and QA reviewers are expected to sign off on a release before it reaches real entrepreneurs.

7.2.3 Production Environment

Purpose: the live system serving real Women Entrepreneurs, Certified Mentors and Government Officers. Deployment to Production requires: (a) a successful Staging deployment and smoke test, (b) a manual approval from a designated release manager recorded in the GitHub Actions Environment protection rule (capturing who approved the release and when, satisfying the audit-logging cross-cutting concern), and (c) execution of the blue-green cutover procedure described below rather than an in-place restart.

7.3 Blue-Green Production Deployment and Rollback

Because the portal's Business Profile, Funding Application and Mentorship Request entities are stateful (Section 8.4 of Assessment Tool 1) and cannot tolerate an in-place restart that drops in-flight requests, Production releases use a blue-green deployment model.

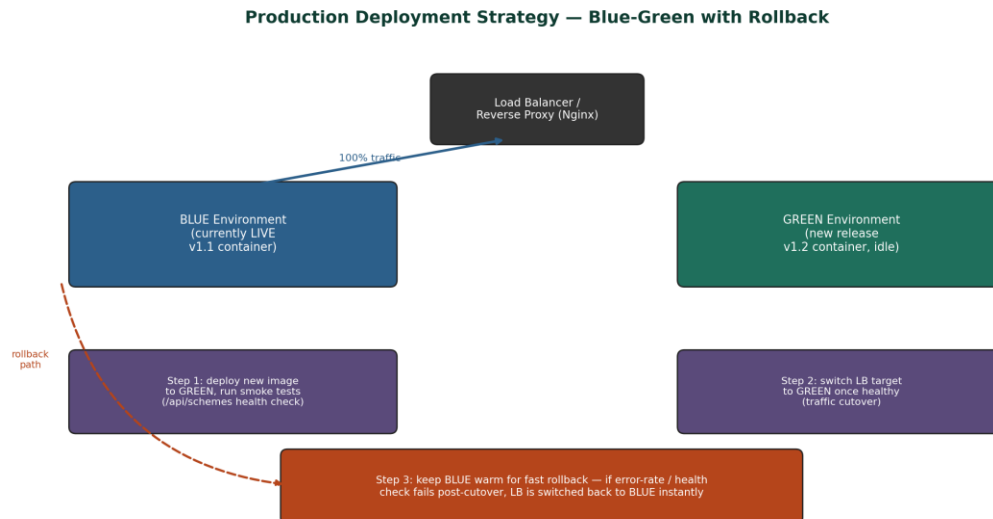


Fig. 3: Blue-green production deployment showing the new release deployed idle alongside the live version, a health-checked traffic cutover, and an immediate rollback path if the new version misbehaves.

- Step 1: the new image is deployed to an idle “Green” container set, side-by-side with the currently live “Blue” set. Automated smoke tests (the same subset used in Staging) run against Green directly, plus the Docker HEALTHCHECK against /api/schemes (NFR7) must report healthy.
- Step 2: once Green is confirmed healthy, the Nginx reverse proxy's upstream target is switched from Blue to Green. This cutover is near-instantaneous and does not require restarting the database container, so in-flight funding-application or mentorship-request submissions are not interrupted.
- Step 3: Blue is kept running, untouched, for a defined “soak” period (e.g., 30 minutes). If error-rate or health-check monitoring detects a problem with Green during this window, the reverse-proxy target is switched back to Blue

immediately — a rollback measured in seconds rather than the many minutes a rebuild-and-redeploy rollback would take.

- Step 4: only after the soak period passes without incident is Blue decommissioned, and it becomes the target for the next release's cutover test.

7.4 Rollback Mechanism (Summary)

Three complementary rollback mechanisms are built into the pipeline:

- Immediate traffic rollback: the blue-green switch described above, used for any issue detected within the soak period.
- Image-tag rollback: because every image is immutably tagged with its Git commit SHA and pushed to GHCR, any previously released version can be redeployed by re-pointing the Docker Compose / deployment manifest at an older tag — no rebuild is ever required, and the last five production tags are always retained in the registry.
- Database rollback safety: all schema migrations applied to the sql.js data layer are written as additive, backward-compatible changes (new nullable columns/tables only) within a release, with an explicit down-migration script and a pre-migration backup of the SQLite file taken automatically by the deployment job, so that a Production rollback to a previous image is never blocked by an incompatible database schema.

8. Security, Quality Checks, Notifications and Rollback in the Pipeline

This section consolidates the security controls, quality gates and communication mechanisms that operate as cross-cutting concerns across all eight pipeline stages, rather than being confined to a single point in the pipeline (as shown at the bottom of Fig. 1).

8.1 Security Controls

Control	Pipeline Stage	What It Protects Against
Branch protection + mandatory PR review	Source Code Management	Unreviewed or unauthorized changes reaching shared branches (supports NFR5, NFR9).
Gitleaks secret scanning	Code Quality & Security Analysis	JWT signing keys, DB passwords or API tokens committed to the repository.
CodeQL SAST	Code Quality & Security Analysis	Injection flaws, authentication-bypass patterns, unsafe deserialization in the Express codebase.
npm audit / Dependabot	Code Quality & Security Analysis	Known-CVE vulnerabilities in third-party npm packages.
Trivy container image scan	Packaging	OS-level and application-level vulnerabilities baked into the Docker image before it is ever pushed.
GitHub Encrypted Secrets	All stages	Hardcoding of JWT_SECRET, DB credentials or registry tokens in workflow files or source code.
RBAC integration/E2E tests (TC-37, TC-41–TC-43)	Automated Testing	Regression of role-based access control that would let an Entrepreneur reach Officer/Mentor-only routes.
Manual approval gate (Production Environment protection rule)	Deploy	An unreviewed or accidental deployment reaching the live system used by real entrepreneurs.

Table 5: Security controls mapped to pipeline stages.

8.2 Quality Checks

In addition to the security controls above, the pipeline enforces: a minimum 70% automated test coverage threshold; zero ESLint errors; a Lighthouse performance score threshold on the built client bundle; and an axe-core accessibility check with zero critical violations. Any of these failing marks the pipeline run red and blocks merge/deploy, exactly as a failing security check would.

8.3 Notification Mechanism

A Slack Incoming Webhook posts a message to a dedicated #wesp-deployments channel at four points in every pipeline run: when the pipeline starts, when the Automated Testing stage completes (pass or fail, with a link to the failing test report if applicable), when a Staging deployment completes and smoke tests finish, and when a Production deployment is either awaiting manual approval or has completed its blue-green cutover. If the Slack webhook itself is unreachable, GitHub Actions falls back to emailing the repository's designated release-manager group, so that a Production-affecting failure is never silently missed.

8.4 Audit Logging

Every pipeline run, every approval decision on the Production Environment protection rule, and every image tag ever pushed to GHCR is retained indefinitely in GitHub's own audit log and Actions run history. This gives the government agency operating the portal a complete, tamper-evident record of who approved which release and when — directly supporting the “ethics” emphasis in the CO4 statement, since it makes every change to a system handling sensitive Udyam and funding data traceable to a named, authenticated individual.

9. Pipeline Configuration / Workflow Definition

The listing below is a representative GitHub Actions workflow file (.github/workflows/ci-cd.yml) that implements the architecture from Section 4, the tools from Section 5, the testing strategy from Section 6 and the deployment strategy from Section 7 as a single, version-controlled configuration file. It is presented here as a design artifact to illustrate exactly how each stage is wired together; it has not been executed against the live repository, consistent with the design-only scope of this assignment.

```
name: WESP CI/CD Pipeline

on:
  push:
    branches: [develop, 'release/**', main]
  pull_request:
    branches: [develop, 'release/**', main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18' }
      - name: Install dependencies
        run: npm ci
      - name: Lint
        run: npm run lint
      - name: Unit tests
        run: npm run test:unit -- --coverage
      - name: Integration tests (Supertest)
        run: npm run test:integration
      - name: Build client (Vite)
        run: npm run build
      - name: E2E tests (Playwright)
        run: npm run test:e2e
      - name: Secret scan
        uses: gitleaks/gitleaks-action@v2
      - name: CodeQL analysis
        uses: github/codeql-action/analyze@v3
      - name: Dependency audit
        run: npm audit --audit-level=high

  package-and-push:
    needs: build-and-test
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t ghcr.io/simats/wesp:${{ github.sha }} .
      - name: Scan image (Trivy)
        uses: aquasecurity/trivy-action@master
        with: { image-ref: 'ghcr.io/simats/wesp:${{ github.sha }}' }
      - name: Push to GHCR
        run: docker push ghcr.io/simats/wesp:${{ github.sha }}

  deploy-dev:
    needs: package-and-push
    if: github.ref == 'refs/heads/develop'
    environment: development
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Development (docker compose)
        run: ./deploy.sh development ${{{ github.sha }}
```

```

deploy-staging:
  needs: package-and-push
  if: startsWith(github.ref, 'refs/heads/release/')
  environment: staging
  runs-on: ubuntu-latest
  steps:
    - name: Deploy to Staging
      run: ./deploy.sh staging ${github.sha}
    - name: Smoke test
      run: npm run test:smoke -- --base-url=https://staging.wesp.gov.in

deploy-production:
  needs: deploy-staging
  if: startsWith(github.ref, 'refs/heads/release/')
  environment:
    name: production
    url: https://wesp.gov.in
  runs-on: ubuntu-latest
  steps:
    - name: Blue-Green deploy
      run: ./deploy-bluegreen.sh production ${github.sha}
    - name: Post-cutover health check
      run: curl -f https://wesp.gov.in/api/schemes
    - name: Notify Slack
      if: always()
      run: ./notify-slack.sh "Production deploy ${job.status}: ${github.sha}"

```

Listing 1: Representative GitHub Actions CI/CD workflow definition for the Women Entrepreneurship Support Portal.

9.1 Explanation of the Configuration

- The on: block triggers the workflow on every push and pull request targeting develop, any release/* branch, or main — matching the branch-to-environment mapping in Table 4.
- The build-and-test job runs on a clean, ephemeral GitHub-hosted Ubuntu runner for every trigger, so that Stages 3, 4 and 5 (Build, Testing, Code Quality & Security) always execute even for a pull request that never gets merged, giving reviewers a pass/fail signal before approving.
- The package-and-push job only runs on an actual push (not a pull request), preventing Docker images from being built and pushed for code that hasn't yet been merged.
- The three deploy-* jobs each declare a GitHub Actions environment (development, staging, production); the production environment is configured (outside this YAML, in repository settings) with a required-reviewers protection rule, which is what implements the manual-approval gate described in Section 7.2.3 without any custom scripting.
- deploy-staging depends on package-and-push and runs the smoke-test suite immediately after deployment; deploy-production depends on deploy-staging succeeding, which enforces the “Staging must pass before Production” rule from Table 4 structurally, at the workflow-graph level, rather than relying on a person remembering to check.

10. Workflow Explanation: From Code Commit to Deployment

This section narrates the complete, end-to-end path a single code change takes through the pipeline shown in Fig. 1, tying together every stage, tool and environment described in the preceding sections.

Step 1 — Local Development and Commit

A developer creates a feature/* branch from develop (for example, feature/csv-export), implements the change, and commits it locally. Pre-commit hooks (Husky, running ESLint and a fast unit-test subset) give immediate local feedback before the code ever reaches GitHub.

Step 2 — Push and Pull Request

The developer pushes the branch and opens a pull request against develop. This immediately triggers the build-and-test job: dependencies are installed with npm ci, ESLint runs, unit tests run with coverage, integration tests run against a disposable sql.js database, the Vite production build runs, and the Playwright E2E suite runs against that build. CodeQL, npm audit and Gitleaks run in parallel. Results are posted directly on the pull request within minutes.

Step 3 — Code Review and Merge Gate

A peer reviewer examines the diff while the automated checks run. The pull request cannot be merged until both the human review is approved and every automated check in Step 2 is green, per the quality gate policy in Section 6.6. If any check fails, the developer fixes the issue and pushes again, re-triggering the same checks — there is no path for a red build to be merged.

Step 4 — Merge to develop and Automatic Development Deployment

Once merged, the build-and-test job re-runs on develop itself (defence in depth against a bad merge), then package-and-push builds the Docker image, scans it with Trivy, and pushes it to GHCR tagged with the commit SHA. deploy-dev then automatically redeploys the Development environment with that exact image — typically within 10–12 minutes of the original push.

Step 5 — Release Branch and Staging Deployment

When the team is ready to cut a release, a release/1.x branch is created from develop. Pushing to this branch re-runs the full build-and-test and package-and-push jobs, then deploy-staging deploys the resulting image to the Staging environment and immediately runs the automated smoke-test suite against the live Staging URL. QA reviewers and, where required, the government client review the release on Staging.

Step 6 — Manual Approval for Production

Only once Staging smoke tests pass does deploy-production become eligible to run. GitHub Actions pauses this job and notifies the designated release manager (via the Slack/email notification mechanism in Section 8.3) that a deployment is awaiting approval. The release manager reviews the Staging sign-off and test results, then approves the deployment directly in GitHub — an action that is permanently recorded in the audit log described in Section 8.4.

Step 7 — Blue-Green Production Cutover

Once approved, deploy-production executes the blue-green procedure from Section 7.3: the new image is started as the idle Green set, health-checked against /api/schemes, and only then does the Nginx reverse proxy cut live traffic over to it. The previous Blue set remains warm for the soak period, ready for an instant rollback.

Step 8 — Post-Deployment Monitoring and Notification

After cutover, the Docker HEALTHCHECK and Uptime Kuma continue to monitor the live endpoint. A final Slack notification confirms successful deployment, including the commit SHA and the approving release manager's identity, closing the loop from the original commit in Step 1 to a verified, monitored, rollback-ready production release.

This complete path — commit → automated build/test/scan → peer review → Development deploy → Staging deploy +

smoke test → manual approval → blue-green Production cutover → monitored soak period — is the concrete mechanism through which the 53 test cases and security controls designed in Assessment Tool 1 are continuously and automatically re-verified for the lifetime of the Women Entrepreneurship Support Portal.

11. Reference Screenshots: Existing Repository and Container Artifacts

The screenshots below are reproduced from the same previously submitted Women Entrepreneurship Support Portal technical/project materials referenced in Assessment Tool 1 (Appendix A), and are provided here as evidence that the CI/CD design in this report is grounded in artifacts (Dockerfile, docker-compose.yml, a running, healthy container) that already exist for this system, rather than being a purely theoretical exercise. These are reference screenshots of repository and container state, not screenshots of an executed CI/CD pipeline run — no GitHub Actions run has been triggered for this assignment.

```
kirank@KIRANS_PC #ING64 ~/.gemini/antigravity/scratch/women-entrepreneurship-portal (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .env.example
    new file:   .gitignore
    new file:   DB_CONNECTION.md
    new file:   Dockerfile
    new file:   FINAL_TEST_REPORT.md
    new file:   README.md
    new file:   REQUIREMENTS_TRACEABILITY.md
    new file:   TEST_CASES.md
    new file:   VIVA_GA.md
    new file:   client/index.html
    new file:   client/src/App.jsx
    new file:   client/src/components/DemoToolbar.jsx
    new file:   client/src/components/DocumentViewerModal.jsx
    new file:   client/src/components/Footer.jsx
    new file:   client/src/components/FundingApplicationModal.jsx
    new file:   client/src/components/Modal.jsx
    new file:   client/src/components/Navbar.jsx
    new file:   client/src/components/ProtectedRoute.jsx
    new file:   client/src/components/SearchFilterBar.jsx
    new file:   client/src/components/Sidebar.jsx
    new file:   client/src/components/StatusCard.jsx
    new file:   client/src/components/StatusBadge.jsx
    new file:   client/src/context/AuthContext.jsx
    new file:   client/src/context/NotificationContext.jsx
    new file:   client/src/context/ToastContext.jsx
    new file:   client/src/index.css
    new file:   client/src/main.jsx
    new file:   client/src/pages/AboutPage.jsx
    new file:   client/src/pages/LandingPage.jsx
    new file:   client/src/pages/LoginPage.jsx
    new file:   client/src/pages/NotFoundPage.jsx
    new file:   client/src/pages/PublicEvents.jsx
    new file:   client/src/pages/PublicFunding.jsx
    new file:   client/src/pages/PublicSchemes.jsx
    new file:   client/src/pages/PublicTraining.jsx
    new file:   client/src/pages/RegisterPage.jsx
    new file:   client/src/pages/UnauthorizedPage.jsx
    new file:   client/src/pages/entrepreneur/AnnouncementPage.jsx
    new file:   client/src/pages/entrepreneur/BusinessPlanPage.jsx
    new file:   client/src/pages/entrepreneur/EntrepreneurDashboard.jsx
    new file:   client/src/pages/entrepreneur/EntrepreneurFunding.jsx
    new file:   client/src/pages/entrepreneur/EntrepreneurSchemes.jsx
    new file:   client/src/pages/entrepreneur/FundingApplicationModal.jsx
    new file:   client/src/pages/entrepreneur/MentorDirectoryPage.jsx
    new file:   client/src/pages/entrepreneur/MyApplicationPage.jsx
    new file:   client/src/pages/entrepreneur/MyEventsPage.jsx
    new file:   client/src/pages/entrepreneur/MyMentorshipRequestsPage.jsx
    new file:   client/src/pages/entrepreneur/MyTrainingPage.jsx
    new file:   client/src/pages/entrepreneur/NetworkingEventsPage.jsx
    new file:   client/src/pages/entrepreneur/ProfilePage.jsx
    new file:   client/src/pages/entrepreneur/TrainingProgramPage.jsx
    new file:   client/src/pages/mentor/MentorDashboard.jsx
    new file:   client/src/pages/mentor/MentorProfilePage.jsx
    new file:   client/src/pages/mentor/MentoredEntrepreneurPage.jsx
    new file:   client/src/pages/mentor/MentoringSessionsPage.jsx
    new file:   client/src/pages/mentor/MentorshipRequestsPage.jsx
    new file:   client/src/pages/officer/AnnouncementManagementPage.jsx
    new file:   client/src/pages/officer/ApplicationManagementPage.jsx
    new file:   client/src/pages/officer/EntrepreneurManagementPage.jsx
    new file:   client/src/pages/officer/FundingManagementPage.jsx
    new file:   client/src/pages/officer/MentorManagementPage.jsx
    new file:   client/src/pages/officer/NetworkingManagementPage.jsx
    new file:   client/src/pages/officer/ReportsPage.jsx
    new file:   client/src/pages/officer/SchemeManagementPage.jsx
    new file:   client/src/pages/officer/TrainingManagementPage.jsx
    new file:   client/src/pages/officer/UserManagementPage.jsx
    new file:   docker-compose.yml
    new file:   docs/ANNEXURE_A_GIT_DOCKER_ASSIGNMENT.md
    new file:   docs/ANNEXURE_A_SCREENSHOTS.md
    new file:   docs/GIT_DOCKER_COMMANDS.md
    new file:   docs/GIT_GITHUB_FINAL_CHECKLIST.md
    new file:   docs/GIT_SCREENSHOT_COMMANDS.md
    new file:   docs/GIT_SCREENSHOT_GUIDE.md
    new file:   package-lock.json
    new file:   package.json
    new file:   server/db/database.js
    new file:   server/db/initSchema.js
```

Fig. 4: git status output showing the single-repository structure (client/, server/, Dockerfile, docker-compose.yml) that Stage 1 (Source Code Management) and Stage 3 (Build) of the pipeline operate on.

```
MINGW64: c:/Users/kiran/Downloads/vaxcare
kiran@KIRANS_PC MINGW64 ~/Downloads/vaxcare (develop)
$ docker compose version
Docker Compose version v5.4.0

kiran@KIRANS_PC MINGW64 ~/Downloads/vaxcare (develop)
$ docker compose config
time="2026-08-18T08:34:21+05:30" level=warning msg="C:\\Users\\kiran\\Downloads\\vaxcare\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
name: vaxcare
services:
  backend:
    build:
      context: C:\\Users\\kiran\\Downloads\\vaxcare
      dockerfile: Dockerfile
    container_name: vaxcare_backend
    depends_on:
      db:
        condition: service_healthy
        required: true
    environment:
      DB_HOST: db
      DB_NAME: vaxcare_db
      DB_PASSWORD: vaxcare_pass
      DB_PORT: "3306"
      DB_USER: vaxcare_user
      PORT: "5000"
      SECRET_KEY: vaxcare_docker_secret_key_2026
    networks:
      vaxcare_network: null
    ports:
      - mode: ingress
        target: 5000
        published: "5000"
        protocol: tcp
    restart: always
  db:
    container_name: vaxcare_mysql
    environment:
      MYSQL_DATABASE: vaxcare_db
      MYSQL_PASSWORD: vaxcare_pass
      MYSQL_ROOT_PASSWORD: vaxcare_root_pass
      MYSQL_USER: vaxcare_user
    healthcheck:
      test:
        - CMD
        - mysqladmin
        - ping
        - -h
        - localhost
        - -u
        - root
        - -pvaxcare_root_pass
      timeout: 5s
      interval: 5s
      retries: 10
```

Fig. 5: docker compose config output resolving the backend service's build context, environment variables and healthcheck-driven dependency on the database service — the same configuration structure that Stage 6 (Packaging) and the Development/Staging Docker Compose deployments in Section 7.2 build upon.

```

PS C:\Users\kiran\Downloads\women-entrepreneurship-portal> docker compose ps
time="2026-08-19T22:28:24+05:30" level=warning msg="C:\\Users\\kiran\\Downloads\\women-entrepreneurship-portal\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                                     COMMAND
SERVICE            CREATED        STATUS                               PORTS
women_entrepreneurship_portal_app  women-entrepreneurship-portal-portal-app  "docker-entrypoint.s..."
portal-app           19 minutes ago Up 19 minutes (healthy)  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
PS C:\Users\kiran\Downloads\women-entrepreneurship-portal>

```

Fig. 6: docker compose ps output showing the portal's application container in an “Up ... (healthy)” state with ports bound — direct evidence of the Docker HEALTHCHECK mechanism against /api/schemes that Stage 8 (Deploy) and the blue-green cutover in Section 7.3 depend on to gate traffic switching.

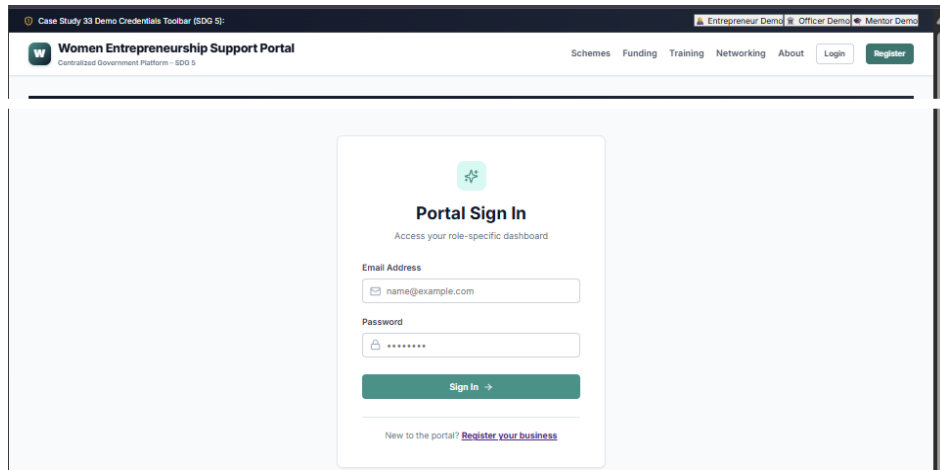


Fig. 7: the portal's Sign In screen, representative of the application that each of the three environments (Development, Staging, Production) serves once a pipeline run reaches Stage 8.

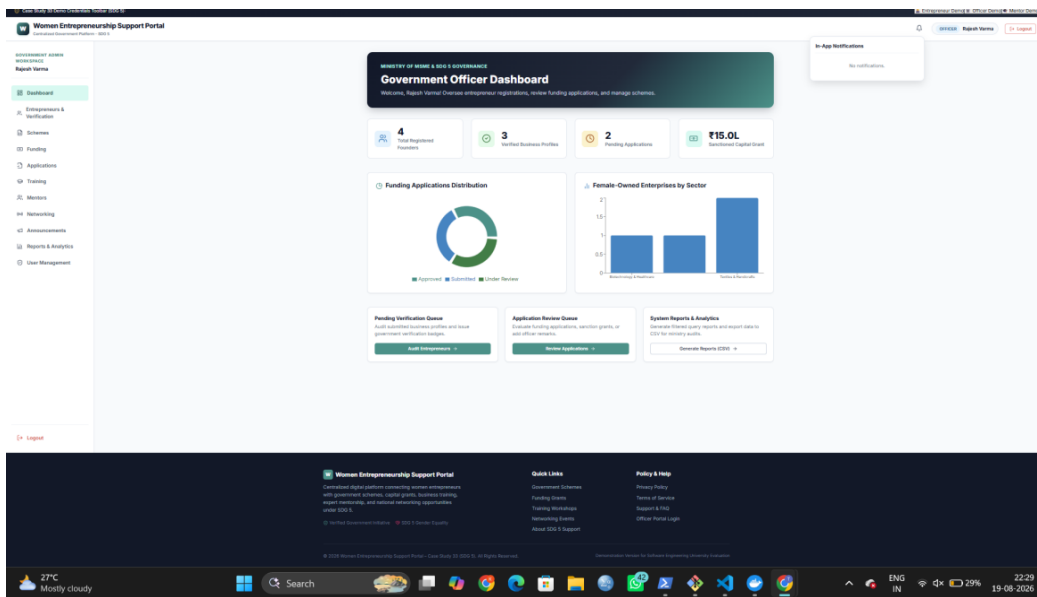


Fig. 8: the Government Officer Dashboard, representative of the live Production environment that the blue-green deployment strategy (Fig. 3) protects from downtime and data inconsistency during a release.

12. Traceability: Requirements to CI/CD Pipeline Stages

To demonstrate that the CI/CD pipeline is not a generic addition but is specifically designed around the Women Entrepreneurship Support Portal's own requirements, the table below extends the Requirements Traceability Matrix from Assessment Tool 1 by mapping each non-functional requirement to the specific pipeline stage(s) and mechanism(s) that verify or enforce it on an ongoing basis.

Req. ID	Requirement	AT1 Coverage Status	CI/CD Stage / Mechanism	New Coverage Status
NFR1	Security	Full (TC-08, TC-37, TC-41–TC-43)	Stage 5 (SAST, dependency & secret scanning) + Stage 4 (RBAC integration/E2E tests) on every commit	Full — now continuous, not point-in-time
NFR2	Performance	Partial (observed indirectly)	Stage 4 (Lighthouse performance budget gate)	Strengthened to an enforced automated gate
NFR3	Usability	Full (TC-48, TC-49)	Stage 4 (E2E journeys assert navigation/validation clarity)	Full — automated
NFR4	Reliability	Full (TC-44–TC-46)	Stage 8 (blue-green cutover, health-checked rollback)	Full — with automatic rollback
NFR5	Maintainability	Partial (architecture review only)	Stage 5 (automated layering/dependency lint rule)	Strengthened to an enforced automated gate
NFR6	Scalability	Partial (schema review only)	Stage 6/8 (containerized, horizontally replicable image; documented Kubernetes migration path)	Design-level, now deployment-ready
NFR7	Availability	Full (TC-44)	Stage 8 (Docker HEALTHCHECK against /api/schemes gates every cutover)	Full — enforced automatically before every release
NFR8	Accessibility	Partial (manual audit recommended)	Stage 4 (axe-core automated accessibility check)	Strengthened to an enforced automated gate
NFR9	Data Integrity	Full (TC-03, TC-29, TC-32, TC-45, TC-47)	Stage 4 (integration tests re-verify unique constraints on every commit)	Full — now continuous
NFR10	Responsiveness	Not Covered	Stage 4 (Playwright multi-viewport responsive test suite)	Gap closed — now covered

Table 6: Extended Requirements Traceability Matrix showing how the CI/CD pipeline continuously enforces, and in several cases strengthens or closes gaps in, the non-functional requirement coverage first assessed in Assessment Tool 1.

13. Pipeline Metrics and Key Performance Indicators

To evaluate whether the CI/CD pipeline is achieving its purpose once operational, the following DORA-aligned and project-specific metrics are proposed for ongoing monitoring.

Metric	Definition	Target for this Project
Deployment Frequency	How often code is successfully deployed to Production	At least weekly during active development
Lead Time for Changes	Time from commit to successful Production deployment	Under 2 hours for a standard change, excluding approval wait time
Change Failure Rate	Percentage of Production deployments requiring rollback or a hotfix	Below 15%
Mean Time to Recovery (MTTR)	Time from a detected Production incident to full recovery	Under 15 minutes, given the blue-green rollback path
Build/Test Duration	Wall-clock time for the build-and-test job	Under 12 minutes
Automated Test Coverage	Line coverage reported by Jest	At least 70%, trending toward 80%
Test Pass Rate	Percentage of automated test runs that pass without flaky re-runs	Above 98%
Security Findings per Release	Count of high/critical CodeQL, npm audit or Trivy findings per release	Zero at time of Production deployment

Table 7: Proposed pipeline health metrics and targets.

14. Risk Analysis and Mitigation

Designing a CI/CD pipeline also introduces new operational risks that must be explicitly managed, distinct from the functional risks already covered by the Assessment Tool 1 test suite.

Risk	Potential Impact	Mitigation
Flaky end-to-end tests	False-failure builds block legitimate merges, eroding trust in the pipeline	Automatic retry of failed E2E tests once before marking red; quarantine list for known-flaky tests reviewed weekly
Secrets leakage via workflow logs	Exposure of JWT signing key or DB credentials	GitHub Encrypted Secrets with masking in logs; Gitleaks scanning on every commit; least-privilege registry tokens
Downtime during a Production release	Entrepreneurs, mentors or officers unable to access the portal	Blue-green deployment with health-checked cutover (Section 7.3) rather than in-place restart
Database migration failure	Data corruption or loss of Udyam/funding records	Additive, backward-compatible migrations only; automatic pre-migration backup; documented down-migration scripts
Unpatched dependency vulnerability	Exploitable security hole reaches Production	Dependabot automated update PRs + npm audit gate blocking merge on high/critical findings
Manual-approval bottleneck	Release delayed if the sole release manager is unavailable	A defined on-call rotation of at least two authorized approvers for the production environment protection rule
Environment drift (Staging $\neq$ Production)	A release passes Staging but fails in Production	Build-once-deploy-many image promotion; identical Docker Compose topology and environment-variable structure across environments
Over-reliance on automated tests hiding new-feature gaps	A feature with no automated test yet ships an undetected regression	Definition-of-done requires an automated test for every new feature before merge; coverage gate enforces this structurally

Table 8: Risks introduced or mitigated by the CI/CD pipeline design, with corresponding mitigations.

15. Conclusion

This report has designed a complete, eight-stage CI/CD pipeline for the Women Entrepreneurship Support Portal (Case Study 33 – SDG 5), built directly on top of the system architecture, requirements and 53-test-case design already established in the previous technical report and Assessment Tool 1. The pipeline automates source code management, build, a four-layer automated testing strategy, code quality and security analysis, containerized packaging, registry push, and a three-environment (Development / Staging / Production) deployment strategy with a blue-green cutover and multi-layered rollback mechanism for Production.

Every design decision in this report — from choosing GitHub Actions and GHCR because they integrate natively with the project's existing GitHub repository, to designing the four-layer test strategy as a direct automation of the Equivalence Partitioning, Boundary Value Analysis, Decision Table and State Transition techniques from Assessment Tool 1 — was grounded in evidence already available for this specific system rather than in a generic CI/CD template. The extended Requirements Traceability Matrix in Section 12 further shows that this pipeline does not merely re-implement Assessment Tool 1's testing in automated form; it actively closes the NFR10 (Responsiveness) coverage gap and strengthens the partial NFR2, NFR5 and NFR8 coverage that Assessment Tool 1 had flagged as design-level or manual-only.

Implementing this pipeline will materially improve the portal's delivery speed (via automatic Development/Staging deployment), its safety (via mandatory quality/security gates and a health-checked, rollback-ready Production release process), and its auditability (via a permanent record of every pipeline run, test result, security scan and approval decision) — collectively constituting the DevOps foundation required, alongside the testing foundation from Assessment Tool 1, before the portal can be responsibly and continuously operated in support of women entrepreneurs, certified mentors and government officers.

This work directly addresses CO4: “Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics,” by translating the test case design of Assessment Tool 1 into a disciplined, automated, continuously enforced delivery pipeline, ready for adoption by the development team responsible for the Women Entrepreneurship Support Portal.

16. References

- GitHub Actions Documentation – <https://docs.github.com/actions>
- Docker Documentation – <https://docs.docker.com>
- Docker Compose Documentation – <https://docs.docker.com/compose>
- GitHub Container Registry Documentation – <https://docs.github.com/packages/working-with-a-github-packages-registry/working-with-the-container-registry>
- CodeQL Documentation – <https://codeql.github.com/docs>
- Trivy Documentation – <https://aquasecurity.github.io/trivy>
- Gitleaks – <https://github.com/gitleaks/gitleaks>
- Playwright Documentation – <https://playwright.dev>
- Jest Documentation – <https://jestjs.io/docs/getting-started>
- React Documentation – <https://react.dev>
- Node.js Documentation – <https://nodejs.org/en/docs>
- Express Documentation – <https://expressjs.com>
- Nginx Documentation – <https://nginx.org/en/docs>
- DORA (DevOps Research and Assessment) Metrics – <https://dora.dev>
- United Nations Sustainable Development Goal 5 – <https://sdgs.un.org/goals/goal5>
- Previous submission: CO4 Assessment Tool 1 – Test Case Design Assignment (Case Study 33), referenced throughout this report.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4

Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25